

type-pm-mech2 入門

制約生成, `let` の完全性, `matcher` まで

type-pm-mech2 project

2026 年 8 月 20 日版

基準実装: `main` (本文作成開始時の commit `7fe112f`)

この文書について

この文書は、Lean 4 リポジトリ `type-pm-mech2` の現行実装を、型推論や定理証明に初めて触れる読者にも追えるように説明する。中心となる問いは次の三つである。

1. 式を調べると得られる `target`, `hard`, `pending` とは何か。
2. 多相 `let` で右辺を先に閉じると、なぜ完全性の証明が難しくなるのか。
3. 現行実装はその問題をどう解き、どこまでを証明済みとしているのか。

後半では、Paper 1 用 `fixture` として実装されている再帰的 `multiset matcher`、とくに `multisetWithListArgument = lambda list. multiset` という補助式の意味と、この式が現在の `no-guess` 方針の下で単独では推論に失敗する理由も説明する。

旧リポジトリと隣接する論文について

この文書は `type-pm-mech2` の Lean コードを正本とする。兄弟リポジトリ `type-pm-mech` は旧実装であり、旧 `SourceTyping`, `inferRaw`, `wBridgeCheck`, `terminal audit`（推論後の再検査）を現行体系へ持ち込まない。また、`../type-pm-paper` の TeX には旧実装を説明する記述が残っているため、現行 `mechanization` の API 一覧としては使わない。現状確認には本書、リポジトリ直下の `README.md`, `DESIGN.md`, `CHECKPOINT.md`, および Lean コードを用いる。

目次

この文書について	i
第 1 章 まず全体像をつかむ	1
1.1 何を機械化しているのか	1
1.2 M0 から M5 まで	1
1.3 三つの層を混同しない	2
第 2 章 制約生成：target, hard, pending	3
2.1 Generated という三つ組	3
2.2 関数適用を一つ生成してみる	3
2.3 なぜ pending をすぐ等式にしないのか	4
2.4 小さな例： $((\lambda z. x) 0)$	4
第 3 章 多相 let は何をしているのか	5
3.1 value とは let の右辺のこと	5
3.2 full-cut 方式	5
3.3 PrincipalBlockClosure	5
3.4 一般化と scheme	6
3.5 具体例を最後まで追う	6
3.6 interface equations	6
第 4 章 なぜ let の完全性証明が難しかったのか	8
4.1 実行版は一つを選び、関係版は任意の代表を許す	8
4.2 単純な大域的 rename では直らない	8
4.3 必要なのは文字列一致ではなく意味の一致	9
第 5 章 M2 の最終解法：四つの層	10
5.1 第 1 層：entailed alignment	10
5.2 第 2 層：body を同じ context へ運ぶ有限 rename	10
5.3 第 3 層：visible closure graph	11
5.4 第 4 層：well-formed supply と freshness	11
5.5 whole let の組み立て	11

第 6 章	定理が公開推論へつながるまで	13
6.1	証明の主な鎖	13
6.2	公開 API	13
6.3	負例にも完全性が効く	14
第 7 章	M3 から M5 までの現在地	15
7.1	M3：宣言と通常の呼出し	15
7.2	M4：pattern と matcher	15
7.3	M5：評価と型安全性	15
第 8 章	multisetWithListArgument を理解する	16
8.1	Paper 1 の multiset 定義との対応	16
8.2	三つの式の役割	16
8.3	なぜ lambda 版単独は現在拒否されるのか	17
8.4	設計上の選択肢	17
8.5	別問題：kernel で大きな推論を計算すること	18
第 9 章	コードを読むための案内	19
9.1	最初に読むファイル	19
9.2	検証コマンド	20
第 10 章	まとめと再開地点	21
10.1	M2 について覚えておく四文	21
10.2	次の作業は M2 ではなく M4/M5	21
付録 A	用語集	22
付録 B	旧説明から現行説明への対応	24
版の基準		25

第 1 章

まず全体像をつかむ

1.1 何を機械化しているのか

`type-pm-mech2` は、Type-PM という型体系を Lean 4 で機械化するプロジェクトである。ここで機械化とは、型や推論規則をプログラムとして定義するだけでなく、健全性や完全性のような性質を Lean の定理として検査することをいう。

通常の関数型言語では、型推論は式に `Int` や $\alpha \rightarrow \beta$ のような型を付ける。Type-PM ではさらに `matcher` を扱う。`matcher` は、値をどのように分解・観察できるかを記述する実行時の値である。そのため型にも次の二つの見方が現れる。

`matcher producer` 実際に作られた `matcher` 値の型。本書では概略 $\text{Matcher}_{\kappa\tau}$ と書く。

`matcher slot` `matcher` を使う場所が要求する型。本書では概略 $\text{Slot}_{\kappa\tau}$ と書く。

τ は対象値の型であり、 κ は capability である。capability は、`matcher` が入力値のどの形を観察できるかを表す型情報である。`producer` から `slot` への変換は単なる型等式ではないため、普通の単一化だけでなく checking という別の処理が必要になる。これが後で `pending` を分離する理由である。

1.2 M0 から M5 まで

実装は段階に分けられている。2026 年 8 月 20 日時点の要点は表 1.1 のとおりである。

表 1.1: 実装段階の要約

段階	主題	状態	意味
M0	型, 代入, 局所 checking	完了	<code>matcher</code> を含む基礎的な型合成と checking を検証済み。
M1	lambda, application, 制約 block	完了	独立した型付け, 停止する単一化, 推論の健全性・完全性・主要性を検証済み。
M2	scheme, 多相 <code>let</code>	完了	一般の入れ子 <code>letE</code> を含む M2–M3 source で完全性・受理同値・主要性を検証済み。

段階	主題	状態	意味
M3	data, primitive, signature	一部	Bool/List, constructor, primitive, ifE と有限 signature を実装済み. M4 と結ぶ listing 全体の回帰が残る.
M4	pattern, matcher, fix	一部	最終構文と再帰 dispatcher は実装済み. 大きい fixture の kernel 推論回帰と M4 全域の完全性・主要性が残る.
M5	評価と型安全性	一部	順序付き探索と多くの実行例を実装済み. closure や user-defined matcher を含む一般の型安全性が残る.

「M2 が完了」の正確な意味

M2 の完全性・主要性は、開始 supply が context に対して well-formed である範囲で完了している. supply は新しい型変数名を割り当てるカウンタである. 公開推論は常に `context.initialSupply` から始めるため、この条件を満たす. well-formed でない任意の開始 supply は「未解決の宿題」ではなく、仕様の対象外である.

1.3 三つの層を混同しない

現行実装には、役割の異なる三つの層がある.

1. 制約生成：source 式から `Generated` を作る.
2. 制約の閉包：生成した制約を解き、主要な結果を得る.
3. 独立した型付け：実行可能推論器を定義に含めない関係 `Typing` で、source program が型を持つことを述べる.

健全性は「推論が成功したなら独立した型付けが存在する」と述べる. 完全性は逆に「独立した型付けが存在するなら推論は成功する」と述べる. 主要性は、推論が返す型から代入によって他の型付け結果を得られることを述べる.

第 2 章

制約生成：target, hard, pending

2.1 Generated という三つ組

Lean では一つの式から得られる情報を次の構造にまとめる.

```
structure Generated where
  target    : Ty
  hard      : List Equation
  pending   : List CheckObligation
```

それぞれの意味は次のとおりである.

target (生の結果型)

target は, 式自身を構文に沿って調べた直後の結果型である. まだ代入を適用していないため, 型変数を含んでよい. 「最終的に確定した型」ではなく, 制約 block が返そうとしている型の入口である.

hard (無条件に必要な等式)

hard は, checking 変換の種類に関係なく必ず成り立つ型等式の列である. たとえば関数適用 $f\ x$ では, f の型が「引数型から結果型への関数」でなければならない. この外形は hard 等式に入る.

pending (後で判定する checking 要求)

pending は, ある生の型が期待型として使えるかを後で判定する要求の列である. 本書では $s \Leftarrow e$ と書き, 「source 型 s を expected 型 e として使えるか」と読む. 普通の型等式になる場合もあれば, matcher producer から slot への特殊変換になる場合もある.

2.2 関数適用を一つ生成してみる

関数式 f から

$$\langle t_f, H_f, P_f \rangle$$

が、引数式 x から

$$\langle t_x, H_x, P_x \rangle$$

が得られたとする．新しい型変数 d, r を割り当てると、 $f\ x$ の生成結果は概略

$$\begin{aligned} target &= r, \\ hard &= H_f ++ H_x ++ [t_f = d \rightarrow r], \\ pending &= P_f ++ P_x ++ [t_x \Leftarrow d] \end{aligned}$$

となる．ここで $++$ は列の連結である．

関数の外形 $t_f = d \rightarrow r$ は常に必要なので hard である．一方、引数 t_x を d として使う方法は、単なる一致か matcher 変換かがまだ分からないので pending へ置く．

「 r checks against d 」の読み方

$[r \Leftarrow d]$ は「式の生の型 r を、使用箇所が要求する型 d として使えるか後で調べる」という記録である． $r = d$ と最初から決める記号ではない．同様に $[p \Leftarrow d]$ は、source 型が p である別の式に同じ期待型 d が課されたことを表す．

2.3 なぜ pending をすぐ等式にしないのか

matcher producer と slot の間には、capability を調べる特殊変換がある．生成時点で要求の最外形が十分に分からない場合、特殊変換を推測すると、処理順により別の結果を選ぶ危険がある．現行設計は、まず hard を最も一般的に解き、その結果で checking の種類を分類する．この方針を本書では *no-guess*、すなわち構造を先回りして推測しない方針と呼ぶ．

これはあらゆる意味的な解を探索する方式ではない．hard の主要解で checking 分岐を固定した後、に受理できるものを *branch-stable acceptance* と呼ぶ．branch-stable とは、後から型を具体化しても checking の分岐選択をやり直さない、という意味である．

2.4 小さな例： $((\lambda z. x) 0)$

外側の変数 x の型を p とする．式

$$((\lambda z. x) 0)$$

を生成する． z の型を a 、適用が期待する引数型を d 、結果型を r とすると、

$$\begin{aligned} target &= r, \\ hard &= [a \rightarrow p = d \rightarrow r], \\ pending &= [\text{Int} \Leftarrow d] \end{aligned}$$

となる．hard を解くと $a = d$ かつ $p = r$ である．その後 pending を調べると $d = \text{Int}$ になる．したがって式全体の結果は p 、つまり x と同じ型である．

ここで r は「value の型」そのものを最初から意味していたのではない． r は右辺式の target として新しく割り当てた仮の結果変数であり、制約を解いた後に p と同じだと分かる．

第 3 章

多相 let は何をしているのか

3.1 value とは let の右辺のこと

Lean の `Expr.letE value body` における `value` は、実行時の値一般を指す言葉ではない。次の source 構文で等号の右側にある式 e_1 を指すフィールド名である。

$$\text{let } y = e_1 \text{ in } e_2$$

このとき `value` は e_1 、`body` は e_2 である。

3.2 full-cut 方式

現行実装の `let` は full-cut 方式である。cut は「境界で切る」という意味であり、右辺の制約 block を `body` より先に完全に閉じる。手順は次のとおりである。

1. 外側 context で右辺 `value` を生成する。
2. 右辺の `hard` を解き、`pending` を分類・解決して、主要 closure を得る。
3. closure の代入を外側 context へ適用する。
4. closure の結果型を、更新後 context に自由でない変数について一般化し、`scheme` を作る。
5. その `scheme` を context 先頭へ追加して `body` を生成する。
6. 右辺の内部制約は親 block へ出さず、外側 context への有限な影響だけを `interface equations` として返す。

この実行可能な手順は `Source/Elaboration.lean` の `elaborate` に現れる。一方、独立した関係 `Elaborates.letE` は実行可能推論器を呼ばず、任意の主要 closure とその吸収性の証明を受け取る。

3.3 PrincipalBlockClosure

主要 closure は二段階の解をまとめる。

1. `hard` 制約を飽和させる最も一般的な代入。
2. 残った `pending` から作る残余等式を解く最も一般的な代入。

二つの代入を合成したものが `closure substitution` であり、それを `target` へ適用した型が `closure`

target である。吸収的であるとは、後から同じ制約を解く代入を合成しても、選んだ主要代表が正規化された形で保たれる性質である。

「最も一般的」であっても、変数名の代表が文字どおり一意になるわけではない。等式 $p = r$ は r を p へ消しても、 p を r へ消しても解ける。この小さな自由度が、let 完全性証明の中心問題になる。

3.4 一般化と scheme

一般化は、結果型に現れる変数のうち、更新後 context に自由に現れないものだけを量化する。scheme 内の量化変数は名前ではなく位置で表す。これを bound-index scheme という。たとえば恒等関数の型 $\alpha \rightarrow \alpha$ を空 context で一般化すれば $\forall \alpha. \alpha \rightarrow \alpha$ になる。

一方、右辺の結果型が外側変数 x の型 p なら、 p は context にも現れるため量化しない。let $y = \dots$ で得た y はその外側 x と同じ単相型 p を持つ。

3.5 具体例を最後まで追う

次の式を考える。

```
lambda x.
  let y = ((lambda z. x) 0) in
  ((lambda w. 0) y)
```

前章の計算から、let 右辺の結果型は外側 x の型 p である。右辺を閉じて p は外側 context に自由に現れるため、 y の scheme は量化されず、単相 p になる。body の `(lambda w. 0)` は $q \rightarrow \text{Int}$ という型を持ち、 $y : p$ へ適用することで $q = p$ となる。body の結果型は Int である。したがって lambda 全体の型は

$$p \rightarrow \text{Int}$$

である。top-level で p を一般化して読むなら

$$\forall p. p \rightarrow \text{Int}$$

である。

$\forall a. \text{Int}$ ではなく $\forall a. a \rightarrow \text{Int}$

全体は `lambda x. ...` なので、引数 x を受け取る関数である。body が常に 0 を返しても、lambda の矢印は消えない。したがって主要型は $\forall a. a \rightarrow \text{Int}$ であり、 $\forall a. \text{Int}$ ではない。後者は実質 Int と同じで、関数でない。

3.6 interface equations

右辺を閉じた代入 S が外側 context の自由変数を変更した場合、その影響を body より外へ伝える必要がある。ただし右辺の全 hard と pending を再び外へ出すと full-cut ではなくなる。そこで、context の各自由型変数 x について

$$x = S(x)$$

だけを作る. `capability` 変数についても同様である. これが `Context.interfaceEquations` である. 親の後続代入 T がこの interface を解くことは, `context` に見える各変数について

$$T(x) = T(S(x))$$

が成り立つことと同値である. interface は右辺の制約全部でも, `body` へ渡す checking 要求でもない. 外側から観測できる有限な効果だけである.

最後に `Generated.fromLet effects body` は

$$\begin{aligned} target &= body.target, \\ hard &= effects ++ body.hard, \\ pending &= body.pending \end{aligned}$$

を返す. 右辺 value の `pending` は境界を越えないが, `body` 自身が作った `pending` は外側 block に残る.

第 4 章

なぜ let の完全性証明が難しかったのか

4.1 実行版は一つを選び、関係版は任意の代表を許す

公開推論器は実行可能な `unify` を使い、右辺に対して一つの closure を計算する。しかし関係 `Elaborates.letE` は、条件を満たす任意の主要かつ吸収的な closure を許す。完全性を示すには、関係側がどの正しい代表を選んでも、公開推論器が選ぶ代表へ運べることを示さなければならない。

先ほどの右辺では `hard` から $p = r$ が得られた。次の二つはどちらも正しい主要な向きである。

$$S_L = [r \mapsto p], \quad S_R = [p \mapsto r].$$

S_L では閉じた context は $x : p$, 右辺結果も p である。 S_R では閉じた context は $x : r$, 右辺結果も r である。意味は同じだが、`body` の生成に使う変数名が違う。`interface` も一方では $p = p$ という自明式、他方では $p = r$ という非自明な別名等式になり得る。

4.2 単純な大域的 rename では直らない

最初に考えたいのは「一方の変数名を全体で置き換えればよい」という案である。しかし全単射な変数名変更は、自明式 $x = x$ を別の自明式 $y = y$ へ移すことしかできない。非自明式 $x = y$ へは変えられない。現行リポジトリにはこの失敗を `GlobalRenamingCounterexample` として Lean で保存している。

ほかにも次の強すぎる案が反例で退けられた。

- 常に左側だけへ別名等式を足す。実際には代表選択により足す側が変わる。
- `hard` だけを共通化し、`pending` は文字どおり同じだと要求する。`body` の `pending` にも代表名の差が残る。
- `body` だけを先に rename し、任意の外側 frame で同値だとする。外側制約が変数の同一性を観測できる。

ここで frame とは、生成済み制約 block を穴へ入れる周囲の制約構造である。完全性の帰納法で再利用するには、裸の block だけでなく適切な周囲へ入れても受理を保つ必要がある。

4.3 必要なのは文字列一致ではなく意味の一致

二つの pending が別の変数名を含んでも、周囲の hard を解くすべての代入の下で同じ要求になればよい。これが `EntailedObligationEq H left right` である。 *entailed* は「hard 理論 H から必ず従う」という意味である。

たとえば hard に $p = r$ があれば

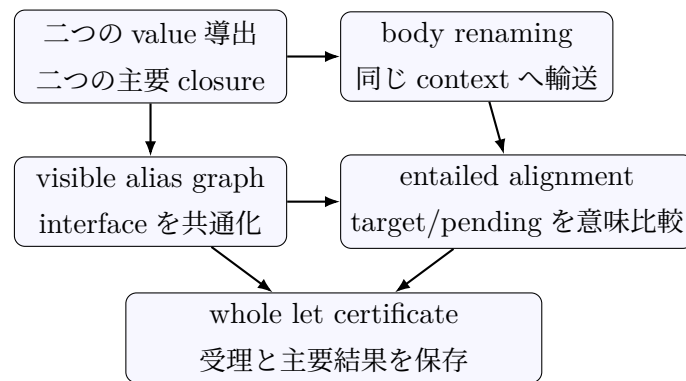
$$p \Leftarrow d \quad \text{と} \quad r \Leftarrow d$$

は文字どおり違うが、 H を解く任意の代入の下では同じになる。この関係は、checking 分岐の分類、hard への昇格、飽和、残余等式、最終受理まで輸送できる。

第 5 章

M2 の最終解法：四つの層

最終証明は、一つの強い rename 定理で全部を済ませない。役割の違う四層を組み合わせる。



5.1 第 1 層：entailed alignment

`EntailedGeneratedAlignment` は、二つの `Generated` について次を要求する。

1. `hard` の解集合が同じである。
2. その `hard` を解く任意の代入の下で `target` が同じになる。
3. 対応する `pending` が同じ要求になる。

さらに `SupportedEntailedAlignmentCertificate` は、終了 `supply`、左右に追加する有限な別名等式、それらを外から観測できなくする `freshness`、`frame` の `support` をまとめる。 `lambda`, `application`, `tuple`, `constructor`, `primitive`, `conditional` は、この証明書を構文に沿って合成できる。

5.2 第 2 層：body を同じ context へ運ぶ有限 rename

二つの closure の結果 context は $x:p$ と $x:r$ のように文字どおり異なり得る。 `body` の帰納仮定を使うには、context を同じにしなければならない。そこで closure の有限 `support` から変数対応 ρ を作る。

ρ には二つの重要な性質がある。

`bodyContext_exact` 左の closed context と一般化 scheme を ρ で移すと、右のものと文字どおり一致する。

`FixesAtOrAbove` value 終了後に新しく割り当てる将来の変数名を ρ が変更しない。

後者により、左 body の導出を同じ source AST, 同じ数値 supply のまま右 context へ輸送できる。これは source の de Bruijn index を変える操作ではない。型変数と capability 変数の有限な名前変更である。

5.3 第 3 層：visible closure graph

body を運ぶ rename だけでは、外側 `fromLet` の interface 差を消せない。そこで二 closure の変数対応のうち、closed context から実際に観測でき、かつ名前が動く辺だけを有限 graph として取り出す。これを visible closure graph という。

必要な側へ fresh な別名等式を追加すると、左右の interface は共通の hard 理論

$$I_L ++ I_R$$

を表すようになる。別名等式は左右非対称でよい。一つの全体 swap を要求しない。

先ほどの例で、一方が $p = p$, 他方が $p = r$ なら、欠けている側へだけ fresh な別名を追加して共通理論へ移す。ただし context や body で既に使われている端点を fresh だと呼んではならない。そのため graph は全 closure support ではなく、context から見える移動辺に限定する。

5.4 第 4 層：well-formed supply と freshness

freshness は「この変数名はたまたま違う」という主張ではない。source 生成変数の由来と数値区間を使って証明する。開始 supply が context に対して well-formed なら、source が新しく作る変数は半开区間 $[start, finish)$ にあり、古い context 変数と衝突しない。

closure の局所性により、block support になく変数は closure substitution が変更しない。これらを合わせると次が言える。

- graph alias の fresh 端点は value の割当区間内にある。
- 対応する側の interface と body context にはその端点が現れない。
- value 区間の alias と、後続 body が作る alias は衝突しない。
- body 用 rename は body 以後の fresh 名を固定する。

5.5 whole let の組み立て

body の証明書は rename 後の座標で作られている。そのため左 body 側の alias 列を ρ の逆向きにたどり、元の座標へ pull back する。pull back とは、移した名前を元の名前空間へ戻す操作である。

最後に、左右の graph alias, pull back した body alias, body の entailed alignment をまとめて、元の二つの `Generated.fromLet` 全体に対する `SupportedEntailedAlignmentCertificate` を構成する。

この手順が重要なのは、反例で偽になった「孤立した body rename は任意の周囲で無害」という中間命題を仮定しない点である。value 導出, closure, transport 済み body 導出を保持したまま, interface と body を一緒に組み立てる。

第 6 章

定理が公開推論へつながるまで

6.1 証明の主な鎖

現行の無条件な M2 証明入口は `Source/FullM2Completion.lean` に集約されている。大きな流れは次のとおりである。

1. `fullM2LetGraphAliasPresentationComplete` が `visible graph presentation` を構成する。
2. `fullM2LetSupportedAssemblyBridge` が `whole-let` 証明書を構成する。
3. `fullM2CoherenceComplete` が通常構文の帰納法と `let` の場合を統合する。
4. `FullM2Replay` が任意の関係的導出を実行可能な導出へ再生する。
5. `wellFormedElaborationPrincipalityComplete` が完全性と主要性をまとめる。

ここで *coherence* は、同じ `source` に対する異なる正しい導出が、受理と主要結果について整合することをいう。*replay* は、関係的導出を入力として、公開推論器が実際に選ぶ `closure` を使った導出を再構成することである。

6.2 公開 API

最終的に次の定理が公開される。

定理	初学者向けの読み方
<code>Typing.infer_isSome</code>	独立した型付けがあれば、公開推論は失敗しない。
<code>Inference.typable_iff_infer_isSome</code>	<code>well-formed signature</code> の下で、型が付くことと推論成功が同値。
<code>Inference.typableDecidable</code>	型が付くかどうかを公開推論で決定できる。
<code>Inference.infer_success_principalResult</code>	推論が返した型は、すべての型付け結果の主要型である。
<code>Inference.infer_principalResult</code>	関係側の主要な型付けから、公開推論の主要結果を得られる。
<code>PrincipalTyping.finiteRenamingEq</code>	二つの主要代表は、有限個の変数名変更を除いて一致する。

`Typing.infer_isSome` と `finiteRenamingEq` は `signature` の `well-formedness` を仮定しない。推論成功から関係的型付けを得る健全性まで使う API は、`Signature.WellFormed` を明示的に受け取る。

6.3 負例にも完全性が効く

`M2Regression.cutRejectsBackflow` は、`let` 右辺を閉じた後に、`body` から互換でない要求を右辺へ逆流させようとする例である。実行可能推論が `none` を返すだけなら、推論器が不完全で見逃した可能性が残る。M2 完全性により、もし独立した `Typing` が存在すれば推論は成功するはずである。実際には失敗するので、`cutRejectsBackflow_not_typable` として非型付けまで証明できる。

第 7 章

M3 から M5 までの現在地

7.1 M3：宣言と通常と呼出し

M3 では Bool/List の data 型, data constructor, pattern constructor, primitive, 有限 signature を追加した. constructor と primitive の引数処理は, 通常の間数適用 `Generated.fromApp` を左から畳み込む. したがって M2 で証明した通常 application の構造を再利用できる.

`add`, `append`, `member`, `deleteFirst`, `map` の canonical scheme, `ifE`, 名前・引数数・型不一致の正負回帰は実装済みである. M3 が表で「一部」なのは, M4 の pattern/matcher と合わせた論文 listing 全体の静的回帰が残るためである.

7.2 M4：pattern と matcher

source 構文は `fixE`, `matcher`, `matchAll`, `matchFirst`, `value pattern`, `pattern constructor`, `pattern function application` まで持つ. M4 の実行可能 elaboration と関係的 elaboration は同じ構文を再帰的に処理し, 成功から `M4.Typing` を得る健全性を証明済みである.

一方, M2–M3 で完了した「型が付くなら必ず推論成功」と「返却型の主要性」は M4 全域では未証明である. `matcher literal` の static check, `fix` 直下 `matcher` の型形状, `clause` 内部の slot 要求, および `let` での任意 closure 選択を同時に扱う必要があるため, 独立した大きな課題である.

7.3 M5：評価と型安全性

M5 には一般の runtime value, closure, `matcher closure`, 順序付き深さ優先探索, `matchAll` と `matchFirst` の関係的評価と `fuel` 付き評価器がある. `fuel` は再帰計算を進められる回数の上限である. 有限な成功導出について, 実行可能評価の健全性, 完全性, `fuel` 単調性を証明している.

型安全性は閉じた整数と再帰的 tuple, および明示的な callback 仮定の下での built-in `matcher` 断片までである. `closure` の定義環境, `primitive` の一般値上の型保存, `user-defined matcher clause`, `data constructor`, 一般 `pattern function` を同じ契約へ接続する作業が残る. これは標準的な保存・進行の形を持つが, 多相 `let` の scheme と runtime 環境を結ぶ設計を含むため, 小さな機械作業ではない.

第 8 章

multisetWithListArgument を理解する

8.1 Paper 1 の multiset 定義との対応

現行 source AST には, top-level の user-defined 値を名前で参照する module 環境がない. 通常の数値参照は de Bruijn index で表す. de Bruijn index とは, 変数名の代わりに「最も内側の binder から何個外か」を自然数で表す方法である.

Paper 1 の概略

```
def multiset m := matcher { ... uses list m ... }
```

では, list は既に定義済みの library matcher である. Lean fixture では multisetDefinition の外側環境に list が一つあるものとして, 内部から index 2 で参照する. index 0 は fixE の引数 m , index 1 は再帰的な自己 multiset, index 2 が外側の list である.

8.2 三つの式の役割

```
multisetDefinition
  = fixE (matcher multisetClauses)

multisetWithListArgument
  = lambda list. multisetDefinition

closedMultisetDefinition
  = multisetWithListArgument listMatcherDefinition
```

multisetDefinition 自体が, Paper 1 の再帰的 `def multiset m := ...` に対応する. fixE を適用すると, 実行時環境は `argument :: self :: definitionEnvironment` となる.

multisetWithListArgument は Paper 1 に表示される別のアルゴリズムではない. 自由依存である list を lambda 引数へ持ち上げた, closure conversion 風の補助式である. closure conversion とは, 関数が外側から参照する自由な値を, 明示的な環境や引数として渡す変換である.

closedMultisetDefinition は source で定義した listMatcherDefinition を実際に渡し, 空の source context から使える閉じた式にする. 静的確認だけなら, multisetDefinition を既知の list scheme を持つ context で調べてもよい. end-to-end の実行回帰では隠れた primitive 値を使わず, 空

context ・ 空 runtime environment から始めるため、この明示的な適用を置いている。

8.3 なぜ lambda 版単独は現在拒否されるのか

`multisetWithListArgument` の制約生成自体は成功し、hard 制約も解ける。失敗は残った checking 要求の解決で起きる。外側 lambda 引数 `list` の型は概略

$$\text{Slot } \chi \alpha \rightarrow \beta$$

まで hard から分かるが、返り型 β が matcher producer であることは hard だけから決まらない。

clause 内には概略

$$\text{Prod}[\beta, \text{Matcher}(\text{List } \chi)(\text{List } \alpha)] \Leftarrow \text{product-slot}$$

という要求が残る。product から product-slot への特殊変換を選ぶには、product の全成分が構文上すでに matcher だと分かる必要がある。 β が変数のままなので、resolver は普通の等式へ進み、最外 constructor の不一致で失敗する。

型が存在しないのではない

β を適切な matcher 型へ具体化すれば、すべての checking 要求を満たす意味的な解はある。期待される全体型は概略

$$(\text{Slot } \chi \alpha \rightarrow \text{Matcher}(\text{List } \chi)(\text{List } \alpha)) \rightarrow \text{Slot } \chi \alpha \rightarrow \text{Matcher}(\text{List } \chi)(\text{List } \alpha)$$

である。しかしその形を選ぶには、hard の主要解に現れていない matcher 外形を β へ先回りして与える必要がある。現在の branch-stable/no-guess 仕様はこの推測をしない。

`closedMultisetDefinition` では、実引数 `listMatcherDefinition` の既知の型が外側 lambda の domain へ hard な情報を与え、 β が matcher だと確定する。その後なら同じ product 要求を特殊変換として分類でき、推論が成功する。

この現象は偶然の M4 実装バグではなく、`GeneratedSemanticAcceptanceRegression` にある M1 の最小境界の大規模な実例である。「完全に具体化すれば意味的に checking 可能」と「hard の主要解で分岐を固定した公開手続きが受理する」は同じではない。

8.4 設計上の選択肢

この高階 matcher constructor を source 言語でどこまで受理したいかにより、選択肢が変わる。

1. 既知型 context または明示注釈を使う。no-guess を保つ最も安全な案である。
2. matcher 位置から期待 slot を逆向きに伝える。双方向型付けに近い規則を追加する。どの構文位置が十分な根拠になるかを明示する必要がある。
3. checking 分岐を探索する。より多くの意味的解を受理できるが、branch-stable 設計と主要性の証明を大きく変更する。

現状では、Paper 1 の multiset 本体そのものが無意味なのではない。名前付き top-level 環境を持

たない fixture を閉じるための外側 lambda が、高階 matcher constructor に対する既知の no-guess 境界を露出している、という位置づけである。

8.5 別問題：kernel で大きな推論を計算すること

大きい matcher fixture では `M4.infer` の実行結果を `#eval` で確認できても、Lean kernel の定義簡約だけで同じ等式を `rfl` により証明することが難しい。測度に基づく再帰で定義した pattern 検査や単一化が簡約を遮るためである。

これは前節の branch-stable 拒否とは別問題である。改善するなら、既存の fuel 構造再帰 `M4.elaborateFuel` を solver まで引数化し、pattern 検査・direct-self 検査・単一化にも計算可能な fuel 版を用意する。その成功を公開版へ輸送する定理を証明した後、小さい catch-all, list matcher, closed multiset の順で kernel 回帰を固定するのが自然である。

第 9 章

コードを読むための案内

9.1 最初に読むファイル

ファイル	読める内容
TypePM/Constraints.lean	Equation, CheckObligation, Generated の最小定義.
TypePM/Source/Syntax.lean	M4 までの最終 source AST.
TypePM/Source/Elaboration.lean	実行可能生成と独立した Elaborates, fromApp, fromLet.
TypePM/BlockClosure.lean	右辺 block を閉じる PrincipalBlockClosure.
TypePM/ContextInterface.lean	外側 context への有限な interface equations.
TypePM/Source/M2Regression.lean	多相 identity, 明示的 let, full-cut 負例.
TypePM/Source/ GlobalRenamingCounterexample.lean	一つの大域 rename では足りない具体的反例.
TypePM/ EntailedPendingTransport.lean	pending を hard 理論の全解の下で比較し, 受理を輸送する層.
TypePM/Source/ EntailedAlignment.lean	source block の意味的な比較証明書.
TypePM/Source/ WholeLetEntailedAssembly.lean	interface と pull back した body alias の whole-let 合成.
TypePM/Source/ FullM2GraphAliasPresentation.lean	visible closure graph の source 由来構成.
TypePM/Source/FullM2Coherence.lean	通常構文と let の場合を統合する帰納法.
TypePM/Source/FullM2Replay.lean	関係的導出から実行可能導出への再生.

ファイル	読める内容
TypePM/Source/ FullM2Completion.lean	完了した M2 の公開入口.
TypePM/Source/Paper1Programs. lean	list/multiset matcher の完全な source fixture.
TypePM/ GeneratedSemanticAcceptance Regression.lean	branch-stable 受理と意味的解の差を示す最小例.

9.2 検証コマンド

Lean 全体はリポジトリ直下で次のように検証する.

```
lake build
git diff --check
```

追加公理への依存は TypePM/AxiomAudit.lean で検査する. M2 の公開入口は Lean 標準の `propext`, `Classical.choice`, `Quot.sound` 以外の追加公理に依存しない.

本書は `tex` ディレクトリで次のように生成する.

```
make
```

出力は `type-pm-mech2-guide.pdf` である.

第 10 章

まとめと再開地点

10.1 M2 について覚えておく四文

1. `let` 右辺は full-cut で先に閉じ、右辺 pending を body へ逆流させない.
2. 任意の主要 closure は意味的には同じでも、代表変数と interface が文字どおり違い得る.
3. body は future 名を固定する有限 rename で同じ context へ運び、outer interface は別の visible alias graph で共通化する.
4. pending は文字列一致でなく、hard を解くすべての代入の下で同じかどうかを比較する.

この四層を well-formed supply による freshness が支え、公開推論の完全性・受理同値・主要性へつながる.

10.2 次の作業は M2 ではなく M4/M5

M2 の ill-formed 開始 supply は仕様外であり、完了のために追加証明する項目ではない. 次の静的な主作業は、大きい Paper 1 fixture を kernel で計算可能な形にし、正例の正確な `M4.infer` 結果と `M4.Typeing`, 負例の非型付けを固定することである.

その後、M4 全域の完全性・主要性を別計画として調べる. M5 では closure, data, primitive, user-defined matcher clause, pattern function を一般の実行時型付けと no-stuck へ接続する.

付録 A

用語集

absorbing closure	後続の解を合成しても、選んだ主要代表が正規化された形で保たれる closure.
branch-stable acceptance	hard の主要解で checking 分岐を固定し、後から分岐選択をやり直さない受理概念.
capability	matcher が入力値のどの形を観察できるかを表す型情報.
closure	生成 block の hard と残余 checking を主要に解く代入と結果をまとめたもの. 実行時関数 closure とは別.
coherence	同じ source に対する異なる正しい導出が、受理と主要結果について整合する性質.
entailed	ある hard 理論を満たすすべての代入の下で必ず成り立つこと.
frame	生成済み制約 block を穴へ入れる周囲の制約構造.
freshness	新しく割り当てた変数名が、既存の support と衝突しない性質.
full-cut	let 右辺の制約 block を body より先に完全に閉じる方式.
hard	checking 変換の種類に依存せず必ず満たす型・capability 等式.
interface equations	閉じた let 右辺が外側 context の自由変数へ与えた影響を表す有限な等式列.
matcher producer	実際に作られた matcher 値の型.
matcher slot	matcher を使用する場所が matcher へ課す要求型.
pending	source 型を expected 型として使えるか, hard を解いた後で分類する checking 要求.
principal	ほかのすべての解が代入を通して得られるという意味で最も一般的であること.
replay	関係の導出から、公開推論器が実際に選ぶ closure を用いた導出を再構成すること.
scheme	量化された型変数を持つ多相型. 現行 M2 では量化変数を位置で表す.
supply	新しい型変数名と capability 変数名を割り当てる二つの自然数カウンタ.
support	型, 制約, 代入などに実際に現れる有限な変数集合.
target	制約生成直後の生の結果型. 最終代入の適用前なので型変数を含んでよい.
terminal audit	推論完了後に履歴や状態を再検査する旧方式. 現行 Typing の定義には含まれない.

visible closure graph	二 closure の有限な変数対応のうち，外側 closed context から観測できる移動辺だけを集めた証明用構造.
well-formed supply	context に既に現れる全自由変数より後から fresh 名を割り当てる開始条件.

付録 B

旧説明から現行説明への対応

旧資料で見かける語	現行 type-pm-mech2 での扱い
SourceTyping	使用しない. 現行は実行可能推論器と独立した <code>Source.Typing</code> を定義する.
inferRaw	旧 API. 現行 source 側は <code>elaborate</code> , <code>infer</code> , M4 側は <code>M4.infer</code> を使う.
wBridgeCheck	旧 terminal audit の一部. 現行の宣言的型付け定義には持ち込まない.
TypingInvariant	旧推論履歴の検査. 現行 M2 完全性は <code>entailed alignment</code> , <code>closure alignment</code> , <code>visible graph</code> で証明する.
PublicTheorems.lean	旧リポジトリの索引. 現行の公開 M2 入口は <code>Source/FullM2Completion.lean</code> .

版の基準

本書は 2026 年 8 月 20 日に, `type-pm-mech2` の `main`, 本文作成開始時 commit `7fe112f` を基準に作成した. 文書追加以外の実装変更が入った場合は, `README.md`, `DESIGN.md`, `CHECKPOINT.md`, `TypePM/AxiomAudit.lean` と照合して更新する.